Linear Data Structures Toolkit

Complexity Analysis & Amortized Analysis Report

Data Structures | ETCCDS202 | Unit 2

School of Engineering & Technology, K.R. Mangalam University

Student: Vaibhav | Roll No.: 2501730098

Program: B.Tech CSE (AI/ML) | Semester: 2

April 2025

1. Dynamic Array — Amortized Analysis of append()

1.1 How the DynamicArray Works

The **DynamicArray** class maintains an internal plain list of fixed capacity. When a new element is appended and the array is full, a new array of **double the capacity** is created, all existing elements are copied over, and then the new element is inserted. This doubling strategy is the key to achieving amortized O(1) per append.

1.2 Amortized Analysis — Simple Explanation

Think of it like a prepaid plan. Every time you do a **cheap append** (no resize needed), you "save up" one credit. When a **resize event** occurs and you have to copy n elements, you use all those saved credits. Because the capacity doubles each time, resizing happens very rarely — at sizes 2, 4, 8, 16, 32... — so the total copying work is spread over many cheap operations.

Formal reasoning (Aggregate Method):

Suppose we perform n appends starting from capacity 1. Resizing happens at sizes 1, 2, 4, 8 ... and copies 1+2+4+...+(n/2) < 2n elements in total. So total work = O(n) for all n appends → average (amortized) cost per append = O(n)/n = **O(1)**.

Operation	Time Complexity	Explanation
append() — no resize	O(1)	Simply places element at next slot
append() — with resize	O(n)	Copies all n elements to new array
append() — amortized	O(1)*	Resize is rare; averaged over all appends
pop()	O(1)	Remove last element, no shifting needed

** Amortized O(1) means the average cost per operation over a long sequence is O(1), even though individual resize operations cost O(n).*

1.3 Observed Resize Events in Test Run

Starting with capacity 2 and appending 10 elements:

```
append(3) → RESIZE: capacity [1] → 4 append(5) → RESIZE: capacity 4 → 8 append(9)
→ RESIZE: capacity 8 → 16
```

Resizes happen at elements 3, 5, and 9 — confirming the doubling pattern (at 2, 4, 8). All other appends are O(1). This matches the theoretical analysis perfectly.

---

[1] .1 Singly Linked List

A **Singly Linked List** is a chain of nodes where each node stores data and a pointer to the next node. There is no index-based access — you must traverse from head.

2. Linked List Operations — Complexity Analysis

Operation	Time Complexity	Explanation
insert_at_beginning()	O(1)	Just update head pointer — no traversal
insert_at_end()	O(n)	Must walk to the last node first
delete_by_value()	O(n)	Must scan the list to find the value
traverse()	O(n)	Visit every node once

2.2 Doubly Linked List

A **Doubly Linked List** extends SLL by adding a **prev** pointer in each node. This allows traversal in both directions and simplifies deletion when a node reference is known (no need to track the previous node separately).

Operation	Time Complexity	Explanation
insert_after_node(target, x)	O(n)	Search for target takes O(n)
delete_at_position(pos)	O(n)	Walk to position index — O(n) in worst case
Head/tail deletion	O(1)	Direct pointer updates with prev/next links

3. Stack & Queue — Complexity Analysis

3.1 Stack ADT (LIFO) using Singly Linked List

The Stack is implemented on top of SinglyLinkedList. **push and pop happen at the head** (beginning of the list), which avoids traversal and gives O(1) time for both.

Operation	Time Complexity	Explanation
push(x)	O(1)	insert_at_beginning — just update head pointer
pop()	O(1)	Remove head — just move head to next node
peek()	O(1)	Return head.data — no traversal needed

3.2 Queue ADT (FIFO) using Singly Linked List

The Queue maintains both a **head** pointer (for dequeue / front) and a **tail** pointer (for enqueue). This way both operations are O(1) — no traversal needed at all.

Operation	Time Complexity	Explanation
enqueue(x)	O(1)	Insert at tail — direct pointer update
dequeue()	O(1)	Remove head — direct pointer update

front()	O(1)	Return head.data — no traversal needed

If we had used only one pointer (head), enqueue would require O(n) traversal. Maintaining head + tail is the standard trick for O(1) Queue operations.

4. Balanced Parentheses Checker — How It Works

4.1 Algorithm

The checker uses our custom **Stack** (not Python's built-in list). It scans each character of the input string:

```
• If opening bracket ( { [ → push onto Stack • If closing bracket ) } ] → pop from Stack and check if it matches • If mismatch or pop from empty Stack → return False • At the end, if Stack is empty → return True (balanced)
```

4.2 Complexity

Operation	Time Complexity	Explanation
is_balanced(expr)	O(n)	One pass through string; push/pop are O(1) each
Space complexity	O(n)	Stack can hold at most n/2 opening brackets

4.3 Test Results

Operation	Time Complexity	Explanation
([])	True (Balanced)	Valid nesting
([)]	False (Not Balanced)	Wrong closing order
(((	False (Not Balanced)	Unclosed brackets remain in Stack
""	True (Balanced)	Empty string is trivially balanced
{[()]}	True (Balanced)	Multiple nested types — all matched
{[(])}	False (Not Balanced)	Inner] closes (incorrectly
)	False (Not Balanced)	Closing bracket with empty Stack

5. Summary — All Operations at a Glance

Data Structure	Operation	Time	Space
DynamicArray	append() amortized	O(1)*	O(n)
DynamicArray	pop()	O(1)	O(1)
SinglyLinkedList	insert_at_beginning	O(1)	O(1)

SinglyLinkedList	insert_at_end	O(n)	O(1)
SinglyLinkedList	delete_by_value	O(n)	O(1)
DoublyLinkedList	insert_after_node	O(n)	O(1)
DoublyLinkedList	delete_at_position	O(n)	O(1)
Stack	push / pop / peek	O(1)	O(1)
Queue	enqueue / dequeue / front	O(1)	O(1)
ParenChecker	is_balanced(expr)	O(n)	O(n)

** Amortized O(1) — individual resize events cost O(n), but when averaged over all appends the per-operation cost is O(1).*

6. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. [Chapter 10 — Elementary Data Structures; Chapter 17 — Amortized Analysis]
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley.
3. GeeksforGeeks — Amortized Analysis for Dynamic Array. https://www.geeksforgeeks.org/analysis-algorithm-set-5-amortized-analysis-introduction/
4. Python Official Documentation — Data Model. https://docs.python.org/3/
5. Course notes: Data Structures (ETCCDS202), K.R. Mangalam University, Unit 2.